

POLINES · ADMINISTRASI BISNIS · ABT-3D

# Lembar Kerja Minggu 6

## Basis Data SQLite dan Operasi Data Dasar (CRUD)

Pemograman Bisnis & Perancangan Sistem Administrasi · Senin 90 menit: membangun database. Selasa 90 menit: CRUD dari Python.

|             |  |               |  |
|-------------|--|---------------|--|
| Nama        |  | NIM           |  |
| Tema proyek |  | Nama file .db |  |

**Petunjuk.** Ikuti langkah sesuai urutan dan ketik kode persis seperti contoh. Setiap tahap punya kotak CEK: centang bila hasil Anda sama sebelum lanjut. Pakai data simulasi, jangan data pribadi pelanggan nyata. File akhir: kasir.db, buat\_database.py, uji\_aturan.py, database.py, main.py, dan lembar ini terisi.

### o. Persiapan

1. Buka VS Code, pilih **File → Open Folder...**, lalu buka folder proyek Minggu 5 (yang berisi `main.py` dan `logika_bisnis.py`).
2. Buat folder baru `minggu-06` di dalamnya agar file Minggu 5 tidak tertimpa. Buka folder `minggu-06` itu di VS Code.
3. Pilih **Terminal → New Terminal**. Ketik `python --version` (Windows: `py --version` bila perlu). Harus tampil Python 3.
4. Tidak perlu memasang apa pun: modul `sqlite3` sudah bawaan Python.

**CEK** ☐ Terminal berada di folder `minggu-06` dan menampilkan versi Python 3.

### BAGIAN A · SENIN — Dari rancangan ke tabel yang hidup

## Bekal ERD 1/3 · Peta data, bukan alur program

Baca bekal ini pada kegiatan A1. Seluruh penjelasan tersedia di Minggu 6; Anda tidak perlu membuka modul Minggu 3.

ERD (Entity Relationship Diagram) adalah gambar rancangan data: apa yang dicatat, informasi apa yang disimpan, dan bagaimana catatan saling terhubung. Seperti merancang buku daftar produk dan buku penjualan sebelum mengisinya.

ERD bukan diagram alur. ERD menunjukkan struktur dan hubungan data, bukan urutan klik, menu, atau langkah program.

| Istilah      | Arti mudah                                        | Contoh kasir                    |
|--------------|---------------------------------------------------|---------------------------------|
| Entitas      | Jenis objek/kejadian yang dicatat; menjadi tabel. | produk dan transaksi            |
| Atribut      | Keterangan yang disimpan; menjadi kolom.          | nama, harga, stok               |
| Baris/record | Satu catatan nyata di tabel.                      | Buku Tulis, harga 5000, stok 20 |
| Relasi       | Hubungan antarentitas.                            | Produk tercatat dalam transaksi |

**Entitas ≠ isi baris. “produk” adalah entitas; “Buku Tulis” adalah salah satu baris di dalamnya. Jangan membuat tabel baru untuk setiap nama barang.**

### PK = nomor identitas catatan

Primary Key (PK) membedakan setiap baris: harus unik dan tidak kosong. produk.id\_produk = 1 berarti satu produk tertentu. Nama bukan pilihan PK yang aman karena nama dapat sama atau berubah.

### FK = nomor rujukan ke catatan lain

Foreign Key (FK) menghubungkan tabel. transaksi.id\_produk merujuk produk.id\_produk. FK boleh berulang karena produk yang sama dapat dibeli berkali-kali. Pada contoh ini FK wajib terisi (NOT NULL).

**Nama kolom sama, peran berbeda: id\_produk adalah PK di produk, tetapi FK di transaksi.**

## Bekal ERD 2/3 · Cara membaca hubungan

Kardinalitas menjawab: satu catatan berhubungan dengan berapa catatan di tabel lain? Hubungan ditentukan oleh aturan bisnis, bukan jumlah baris yang kebetulan sudah diisi.

| Hubungan | Arti / contoh                                                                                               |
|----------|-------------------------------------------------------------------------------------------------------------|
| 1:1      | Satu pegawai memiliki paling banyak satu kartu akses aktif; satu kartu milik satu pegawai.                  |
| 1:N      | Satu produk dapat muncul di banyak catatan transaksi; setiap catatan transaksi menunjuk satu produk.        |
| M:N      | Satu nota memuat banyak produk; satu produk muncul di banyak nota. Perlu tabel penghubung detail_transaksi. |

### Contoh ERD praktik minggu ini: hubungan 1:N

| PRODUK                                | TRANSAKSI                                                            |
|---------------------------------------|----------------------------------------------------------------------|
| PK id_produk<br>nama<br>harga<br>stok | PK id_transaksi<br>tanggal<br>FK id_produk<br>jumlah<br>harga_satuan |

### produk.id\_produk (PK) ← transaksi.id\_produk (FK)

Satu produk → 0..N transaksi. Setiap transaksi → tepat 1 produk.

1 = satu; N = banyak; 0..N = boleh belum terjual atau terjual berkali-kali. FK berada di transaksi, yaitu sisi banyak.

Produk yang belum terjual tidak perlu dibuatkan transaksi kosong.

| produk.id_produk (PK) | nama       |
|-----------------------|------------|
| 1                     | Buku Tulis |
| 2                     | Pulpen     |

Contoh data untuk membaca hubungan (bukan data awal tambahan):

| id_transaksi (PK) | id_produk (FK) | jumlah |
|-------------------|----------------|--------|
| 1                 | 1              | 2      |
| 2                 | 1              | 1      |

Dua transaksi menunjuk Buku Tulis ID 1: sah, FK boleh berulang. Pulpen ID 2 belum mempunyai transaksi: juga sah. FK 999 ditolak jika produk 999 tidak ada dan penjaga FK aktif.

Batas contoh: transaksi pada skrip ini mencatat penjualan satu jenis produk, bukan nota lengkap. Nota banyak produk memerlukan transaksi (kepala nota) + detail\_transaksi (baris barang) + produk. Struktur lanjutan itu tidak wajib dibuat pada praktik dasar ini.

## Bekal ERD 3/3 · Dari gambar menjadi tabel SQLite

| Rancangan ERD             | Terjemahan SQLite                                                                 |
|---------------------------|-----------------------------------------------------------------------------------|
| Entitas produk            | CREATE TABLE produk (...)                                                         |
| Atribut nama, harga, stok | nama TEXT; harga INTEGER; stok INTEGER                                            |
| PK produk.id_produk       | id_produk INTEGER PRIMARY KEY AUTOINCREMENT                                       |
| FK transaksi.id_produk    | id_produk INTEGER NOT NULL + FOREIGN KEY (id_produk) REFERENCES produk(id_produk) |
| Aturan isian              | NOT NULL untuk wajib terisi; CHECK untuk nilai yang diperbolehkan                 |

### Urutan merancang

1. Sebutkan dua entitas dan tulis hubungan dengan satu kalimat bisnis.
2. Tentukan atribut dan PK pada setiap entitas.
3. Untuk 1:N, letakkan FK di sisi banyak. ID PK dan FK contoh ini sama-sama INTEGER.
4. Tulis FOREIGN KEY ... REFERENCES ... untuk menyatakan rujukan.
5. Tentukan aturan isian. Harga rupiah dan stok = INTEGER; tanggal = TEXT berformat YYYY-MM-DD.
6. Saat memakai SQLite, aktifkan PRAGMA foreign\_keys = ON di setiap koneksi.

Aturan isian melengkapi ERD, bukan menggantikan hubungan. NOT NULL melarang nilai kosong (NULL); CHECK menjaga nilai, misalnya harga > 0. SQLite perlu PRAGMA agar aturan FK benar-benar dijalankan.

### Cek pemahaman sebelum mengetik

1. “Buku Tulis” itu entitas atau isi satu baris? Mengapa?

Jawaban: .....

2. Mengapa transaksi.id\_produk boleh berulang, tetapi produk.id\_produk tidak?

Jawaban: .....

3. Di tabel mana FK diletakkan untuk hubungan produk–transaksi?

Jawaban: .....

4. Apakah produk boleh belum memiliki transaksi? Jelaskan arti 0..N.

Jawaban: .....

**Lanjutkan A1: pakai contoh produk–transaksi di atas. Untuk proyek lain, buat rancangan dengan pola yang sama; bahan lama tidak menjadi prasyarat.**

**A1. Pahami ERD dan rancang tabel** menit 10–25

Gunakan teori dan contoh ERD di bagian ini; tidak perlu membuka modul Minggu 3. Isi tabel di bawah untuk produk dan transaksi, lalu rancang satu entitas tambahan untuk tema proyek Anda. Baris pertama adalah contoh.

| Tabel     | Primary Key  | Kolom + tipe data                                                     | Foreign Key ke | Aturan (CHECK) |
|-----------|--------------|-----------------------------------------------------------------------|----------------|----------------|
| transaksi | id_transaksi | tanggal TEXT, id_produk INTEGER, jumlah INTEGER, harga_satuan INTEGER | produk         | jumlah > 0     |
|           |              |                                                                       |                |                |
|           |              |                                                                       |                |                |
|           |              |                                                                       |                |                |

**Ingat.** Rupiah disimpan sebagai INTEGER (angka bulat). Tanggal disimpan sebagai TEXT berformat 'YYYY-MM-DD'.

**A2. Buat database dengan buat\_database.py** menit 25–40

1. Salin file buat\_database.py dari dosen ke folder minggu-06, atau buat file baru dan ketik kode berikut.
2. Baca komentar di kode (tulisan setelah tanda # atau --). Komentar tidak dijalankan Python.
3. Simpan (Ctrl+S). Di terminal ketik python buat\_database.py lalu Enter.
4. Jalankan sekali lagi. Tidak boleh ada error, dan data awal tidak bertambah dua kali.

```

buat_database.py
import sqlite3

koneksi = sqlite3.connect("kasir.db")
koneksi.execute("PRAGMA foreign_keys = ON") # wajib: di SQLite FK mati secara bawaan

koneksi.executescript("""
CREATE TABLE IF NOT EXISTS produk (
    id_produk    INTEGER PRIMARY KEY AUTOINCREMENT,
    nama         TEXT     NOT NULL,
    harga        INTEGER NOT NULL CHECK (typeof(harga) = 'integer' AND harga > 0),
    stok         INTEGER NOT NULL DEFAULT 0 CHECK (stok >= 0)
);

CREATE TABLE IF NOT EXISTS transaksi (
    id_transaksi INTEGER PRIMARY KEY AUTOINCREMENT,
    tanggal      TEXT     NOT NULL,           -- format 'YYYY-MM-DD'
    id_produk    INTEGER NOT NULL,
    jumlah       INTEGER NOT NULL CHECK (jumlah > 0),
    harga_satuan INTEGER NOT NULL,           -- harga saat transaksi terjadi
    FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
);
""")

# Data awal hanya diisi bila tabel produk masih kosong,
# sehingga skrip aman dijalankan berkali-kali.
jumlah_produk = koneksi.execute("SELECT COUNT(*) FROM produk").fetchone()[0]
if jumlah_produk == 0:
    koneksi.executemany(

```

```

        "INSERT INTO produk (nama, harga, stok) VALUES (?, ?, ?)",
        [
            ("Buku Tulis", 5000, 20),
            ("Pulpen", 3000, 30),
            ("Map Plastik", 2500, 10),
        ],
    )
    koneksi.execute(
        "INSERT INTO transaksi (tanggal, id_produk, jumlah, harga_satuan) "
        "VALUES (?, ?, ?, ?)",
        ("2026-10-12", 1, 2, 5000),
    )
    koneksi.commit()

koneksi.close()
print("Database dan tabel siap.")

```

| Bagian kode              | Artinya                                                                                                    |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| PRAGMA foreign_keys = ON | Menyalakan penjaga relasi. Di SQLite penjaga ini mati secara bawaan, jadi wajib ditulis di setiap koneksi. |
| IF NOT EXISTS            | Buat tabel hanya bila belum ada, sehingga skrip aman dijalankan berkali-kali.                              |
| CHECK (...)              | Aturan isian. Harga harus angka bulat lebih dari nol; stok tidak boleh negatif.                            |
| FOREIGN KEY              | Setiap transaksi wajib merujuk produk yang benar-benar ada.                                                |
| harga_satuan             | Harga pada hari transaksi, seperti nota. Transaksi lama tidak berubah walau harga produk naik.             |

**CEK** ☐ Terminal menulis “Database dan tabel siap.” dan file kasir.db muncul di panel Explorer.

### A3. Tambahkan satu tabel dari rancangan A1 menit 40–65

1. Pilih satu entitas tambahan yang Anda rancang di A1 (misalnya pemasok). Tentukan atribut, PK, dan hubungannya dengan tabel lain menggunakan panduan ERD di bagian ini.
2. Tulis perintah `CREATE TABLE IF NOT EXISTS ...` di kotak berikut, lalu tambahkan ke dalam tanda `"""` pada `buat_database.py`.
3. Bila tabel itu berelasi, jangan lupa kolom FK dan baris `FOREIGN KEY ... REFERENCES ...`.
4. Jalankan lagi `python buat_database.py`. Tema selain kasir boleh mengganti nama file menjadi `<nama_proyek>.db` di semua skrip.

*Tulis CREATE TABLE tabel tambahan Anda di sini.*

**CEK** ☐ Skrip berjalan tanpa error setelah tabel tambahan dimasukkan.

## A4. Uji penjaga database menit 65–85

Buat file `uji_aturan.py`, ketik kode berikut, lalu jalankan dengan `python uji_aturan.py`. Skrip ini mencoba memasukkan data yang melanggar aturan, lalu membatalkannya (rollback) agar database tetap bersih.

### `uji_aturan.py`

```
import sqlite3

def coba(judul, koneksi, perintah_sql):
    try:
        koneksi.execute(perintah_sql)
        print(judul, "-> DITERIMA")
    except sqlite3.IntegrityError as e:
        print(judul, "-> DITOLAK:", e)

# Koneksi dengan penjaga relasi aktif
koneksi = sqlite3.connect("kasir.db")
koneksi.execute("PRAGMA foreign_keys = ON")
coba("A. Harga berupa teks", koneksi,
     "INSERT INTO produk (nama, harga) VALUES ('Pensil', 'seribu')")
coba("B. Produk tidak ada", koneksi,
     "INSERT INTO transaksi (tanggal, id_produk, jumlah, harga_satuan) "
     "VALUES ('2026-10-12', 999, 1, 5000)")
koneksi.rollback() # batalkan apa pun yang sempat masuk
koneksi.close()

# Koneksi TANPA baris PRAGMA: lihat bedanya
koneksi = sqlite3.connect("kasir.db")
coba("C. Produk tidak ada, tanpa PRAGMA", koneksi,
     "INSERT INTO transaksi (tanggal, id_produk, jumlah, harga_satuan) "
     "VALUES ('2026-10-12', 999, 1, 5000)")
koneksi.rollback() # jangan simpan data yatim hasil uji C
koneksi.close()
```

| Uji | Yang dicoba                   | Hasil yang diharapkan                  | Hasil saya (salin dari terminal) |
|-----|-------------------------------|----------------------------------------|----------------------------------|
| A   | Harga diisi teks 'seribu'     | DITOLAK: CHECK constraint failed       |                                  |
| B   | Transaksi untuk produk ID 999 | DITOLAK: FOREIGN KEY constraint failed |                                  |
| C   | Sama seperti B, tanpa PRAGMA  | DITERIMA                               |                                  |

**Pertanyaan:** mengapa uji C diterima padahal datanya sama dengan uji B? Apa akibatnya bagi laporan bila hal ini terjadi pada data nyata?

---



---



---

## A5. Refleksi Senin

Tanpa aturan CHECK, SQLite mau menyimpan teks “seribu” di kolom angka. Siapa yang sebaiknya menjaga pintu masuk data, program Python atau database? Jelaskan dengan kata-kata Anda.

---

**CEK** ☐ kasir.db, buat\_database.py, dan uji\_aturan.py tersimpan di folder minggu-06 untuk dipakai hari Selasa.

## BAGIAN B · SELASA — Python menyimpan dan mengelola data

Minggu ini kita memisahkan tugas ke file yang berbeda. Kelas jalur Streamlit Minggu 5 membuat database.py yang sama persis; fungsinya dipanggil dari app.py.

| File             | Tugas                                                                  |
|------------------|------------------------------------------------------------------------|
| main.py          | Tampilan terminal: menu, input, dan print                              |
| logika_bisnis.py | Perhitungan dari Minggu 5 (subtotal, diskon). Tidak diubah minggu ini. |
| database.py      | BARU: menyimpan dan mengambil data dari kasir.db                       |

### B1. Tulis database.py menit 10–45

1. Buat file baru database.py di folder yang sama dengan kasir.db.
2. Ketik kode berikut. Perhatikan pola yang selalu sama: buka koneksi → try → execute → commit → finally close.
3. Perhatikan tanda tanya ?. Nilai dikirim sebagai parameter, tidak dirangkai ke teks SQL. Ini mencegah error karena tanda kutip pada nama (misalnya Toko D'Arso) dan mencegah isian jahat.

```
database.py
import sqlite3

NAMA_DB = "kasir.db"

def buka_koneksi():
    koneksi = sqlite3.connect(NAMA_DB)
    koneksi.execute("PRAGMA foreign_keys = ON") # ulangi di setiap koneksi
    return koneksi

def tambah_produk(nama, harga, stok):
    koneksi = buka_koneksi()
    try:
        koneksi.execute(
            "INSERT INTO produk (nama, harga, stok) VALUES (?, ?, ?)",
            (nama, harga, stok),
        )
        koneksi.commit()
    finally:
        koneksi.close() # tetap ditutup walau terjadi error

def daftar_produk():
    koneksi = buka_koneksi()
    try:
        return koneksi.execute(
            "SELECT id_produk, nama, harga, stok FROM produk"
        ).fetchall()
    finally:
        koneksi.close()
```

```
def ubah_harga(id_produk, harga_baru):
    koneksi = buka_koneksi()
    try:
        cursor = koneksi.execute(
            "UPDATE produk SET harga = ? WHERE id_produk = ?",
            (harga_baru, id_produk),
        )
        koneksi.commit()
        return cursor.rowcount    # 0 berarti ID tidak ditemukan
    finally:
        koneksi.close()
```

**Mengapa try / finally?** Bagian finally selalu dijalankan walaupun terjadi error, sehingga koneksi pasti ditutup dan file .db tidak terkunci. Catatan: with sqlite3.connect(...) hanya mengurus commit, tidak menutup koneksi.

## B2. Lengkapi fungsi hapus\_produk menit 45–55

Tulis fungsi hapus\_produk(id\_produk) di bagian bawah database.py. Contoh polanya adalah ubah\_harga. Petunjuk: perintah SQL-nya DELETE FROM produk WHERE id\_produk = ?, parameternya (id\_produk,) (perhatikan koma), dan fungsi mengembalikan cursor.rowcount.

*Tulis fungsi hapus\_produk di sini, lalu salin ke database.py.*

**Awas!** DELETE atau UPDATE tanpa WHERE akan mengenai SEMUA baris di tabel.

## B3. Tulis main.py (menu terminal) menit 25–70

Buat main.py di folder yang sama. Program ini menampilkan menu dan memanggil fungsi di database.py. Dua except di bawah adalah dua lapis penjaga: ValueError untuk isian bukan angka, IntegrityError bila database menolak.

```
main.py
import sqlite3
import database

def tampilkan_produk():
    print("\nID | Nama | Harga | Stok")
    for id_produk, nama, harga, stok in database.daftar_produk():
        print(id_produk, "|", nama, "|", harga, "|", stok)

while True:
    print("\n=== KELOLA PRODUK ===")
```

```

print("1. Tampilkan  2. Tambah  3. Ubah harga  4. Hapus  0. Keluar")
pilihan = input("Pilih menu: ")
try:
    if pilihan == "1":
        tampilkan_produk()
    elif pilihan == "2":
        nama = input("Nama produk: ")
        harga = int(input("Harga (angka bulat, tanpa Rp/titik): "))
        stok = int(input("Stok awal: "))
        database.tambah_produk(nama, harga, stok)
        print("Produk tersimpan.")
    elif pilihan == "3":
        id_produk = int(input("ID produk: "))
        harga_baru = int(input("Harga baru: "))
        if database.ubah_harga(id_produk, harga_baru) == 0:
            print("ID tidak ditemukan.")
        else:
            print("Harga diubah.")
    elif pilihan == "4":
        id_produk = int(input("ID produk yang dihapus: "))
        if input("Yakin hapus? (y/n): ").lower() == "y":
            if database.hapus_produk(id_produk) == 0:
                print("ID tidak ditemukan.")
            else:
                print("Produk dihapus.")
    elif pilihan == "0":
        print("Selesai.")
        break
    else:
        print("Menu tidak dikenal.")
except ValueError:
    print("Harus diisi angka bulat.")
except sqlite3.IntegrityError as e:
    print("Database menolak:", e)

```

Jalankan dengan python main.py. Pilih menu 1 untuk melihat data awal: Buku Tulis (ID 1), Pulpen (ID 2), dan Map Plastik (ID 3).

**CEK** ☐ Menu tampil, dan menu 1 menampilkan tiga produk data awal.

## B4. Uji silang empat skenario menit 70–85

Jalankan setiap skenario berurutan memakai main.py. Setelah setiap skenario, pilih menu 1 untuk memeriksa daftar. Tulis hasil yang benar-benar Anda lihat.

| No | Skenario & langkah                             | Hasil yang diharapkan                           | Hasil saya | Sesuai?                                                    |
|----|------------------------------------------------|-------------------------------------------------|------------|------------------------------------------------------------|
| 1  | Tambah: Penghapus, 2000, stok 15               | "Produk tersimpan.", muncul dengan ID baru      |            | <input type="checkbox"/> Ya <input type="checkbox"/> Tidak |
| 2  | Ubah harga ID 2 (Pulpen) menjadi 3500          | "Harga diubah.", harga Pulpen 3500              |            | <input type="checkbox"/> Ya <input type="checkbox"/> Tidak |
| 3  | Ubah harga ID 999                              | "ID tidak ditemukan."                           |            | <input type="checkbox"/> Ya <input type="checkbox"/> Tidak |
| 4a | Hapus ID 1 (Buku Tulis, sudah punya transaksi) | Database menolak: FOREIGN KEY constraint failed |            | <input type="checkbox"/> Ya <input type="checkbox"/> Tidak |

| No | Skenario & langkah       | Hasil yang diharapkan                           | Hasil saya | Sesuai?                                                    |
|----|--------------------------|-------------------------------------------------|------------|------------------------------------------------------------|
| 4b | Hapus ID 3 (Map Plastik) | "Produk dihapus.", produk lain utuh             |            | <input type="checkbox"/> Ya <input type="checkbox"/> Tidak |
| 5  | Tambah dengan harga abc  | "Harus diisi angka bulat.", program tetap jalan |            | <input type="checkbox"/> Ya <input type="checkbox"/> Tidak |
| 6  | Tambah dengan harga -5   | Database menolak: CHECK constraint failed       |            | <input type="checkbox"/> Ya <input type="checkbox"/> Tidak |

Satu perbaikan yang saya lakukan selama pengujian:

.....

.....

## B5. Refleksi dan jembatan ke Minggu 7

Minggu depan `main.py` diganti form Streamlit. Bagian mana dari pekerjaan hari ini yang menurut Anda tidak perlu diubah sama sekali, dan mengapa?

.....

.....

.....

## Kotak bantuan: pesan error yang sering muncul

| Pesan di terminal                     | Kemungkinan penyebab dan solusi                                                                                                                                          |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| no such table: produk                 | kasir.db dibuat di folder lain atau <code>buat_database.py</code> belum dijalankan. Pastikan terminal di folder yang sama, lalu jalankan <code>buat_database.py</code> . |
| database is locked                    | Ada program lain yang masih membuka <code>kasir.db</code> (misalnya SQLite Viewer atau program yang belum ditutup). Tutup lalu coba lagi.                                |
| No module named 'database'            | <code>database.py</code> tidak satu folder dengan <code>main.py</code> , atau nama file salah ketik ( <code>database.py.txt</code> ).                                    |
| IntegrityError: ... constraint failed | Bukan bug: database menolak data yang melanggar aturan. Baca nama aturan yang disebut.                                                                                   |
| IndentationError                      | Jumlah spasi di awal baris tidak konsisten. Gunakan 4 spasi per tingkat.                                                                                                 |

## Daftar Periksa pengumpulan

|                          |                                                                                                   |
|--------------------------|---------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | kasir.db berisi minimal dua tabel berelasi sesuai ERD (PK/FK/CHECK)                               |
| <input type="checkbox"/> | <code>buat_database.py</code> dan <code>uji_aturan.py</code> berjalan tanpa error                 |
| <input type="checkbox"/> | <code>database.py</code> berisi tambah, daftar, ubah, dan hapus dengan query berparameter ?       |
| <input type="checkbox"/> | <code>PRAGMA foreign_keys = ON</code> ada di setiap koneksi                                       |
| <input type="checkbox"/> | <code>main.py</code> (atau <code>app.py</code> ) berjalan; input salah tidak menghentikan program |
| <input type="checkbox"/> | Tabel A4 dan B4 terisi hasil asli, beserta refleksi                                               |